ZHENGZHOU UNIVERSITY

School of Computer and Artificial Intelligence

Design and Analysis of Algorithms

Lab Practice 02 | Sequential Search and Binary Search

Instructor: Dr. Hussain | Programming language: C++ or Python | Lab type: Hands-on

1. Learning objectives

In this lab, students will learn how searching works, implement two searching methods, trace their execution, and test their programs. Algorithm complexity analysis and Big O notation are not required.

- Explain sequential search and binary search in simple terms.
- Write working programs using either C++ or Python.
- Find a target value and report its zero-based index, or state that it was not found.
- Trace which array values are checked and test different inputs.

2. Task 1: Sequential Search

Write a program that stores 10 integers in an array (or Python list), asks the user for a target number, and checks the elements one by one using a loop. Print “Element Found” with its index, or “Element Not Found”.

Example array: 12, 5, 8, 20, 15, 7, 30, 10, 25, 18

Example input: 20 Expected output: Element Found! Index: 3

3. Task 2: Binary Search

Write a program that uses a sorted array of 10 integers. Ask for a target, calculate the middle index using low and high boundaries, and continue searching in the left or right half until the target is found or the search ends. Print the index or “Element Not Found”.

Example sorted array: 2, 5, 8, 12, 15, 20, 25, 30, 35, 40

Example input: 25 Expected output: Element Found! Index: 6

Important: Binary search requires sorted input. Do not apply it directly to an unsorted array.

Pseudocode reference: Searching algorithms

Pseudocode is a language-independent way to describe an algorithm. Read each step before writing your C++ or Python program. Array indices start at 0.

A. Sequential Search

```
FUNCTION SequentialSearch(A, target)
    FOR i FROM 0 TO length(A) - 1
        IF A[i] == target THEN
            RETURN i
        END IF
    END FOR
    RETURN -1
END FUNCTION
```

B. Binary Search (sorted array only)

```
FUNCTION BinarySearch(A, target)
    low = 0
    high = length(A) - 1
    WHILE low <= high
        mid = low + (high - low) // 2
        IF A[mid] == target THEN
            RETURN mid
        ELSE IF A[mid] < target THEN
            low = mid + 1
        ELSE
            high = mid - 1
        END IF
    END WHILE
    RETURN -1
END FUNCTION
```

A return value of -1 means the target was not found. In binary search, // means integer division. Students should translate the pseudocode into their chosen language and explain each step.

4. Task 3: Manual tracing

Given sorted array: [5, 10, 15, 20, 25, 30, 35, 40]. Target: 30. Trace both algorithms and record each value checked. Use zero-based indices.

Step	Sequential Search	Binary Search
1	5	20
2	10	30 (Found)
3	15	—
4	20	—
5	25	—
6	30 (Found)	—

In your report, explain why the two algorithms examine different values. Create your own tracing table for a different target value as well.

5. Task 4: Modify and test

Run both programs for each of these situations. Record the input, result, and output screenshot:

- The target is the first element.
- The target is the last element.
- The target is not in the array.

6. Questions students should answer

1. What is the difference between sequential search and binary search?
2. Why must the input array be sorted for binary search?
3. Which algorithm performs fewer comparisons for large sorted arrays?
4. What happens when the target element is not present?
5. Can sequential search work on an unsorted array?
6. What happens if binary search is applied directly to unsorted data?
7. Why is binary search considered a divide-and-conquer algorithm?
8. Which algorithm would you choose for a small unsorted dataset, and why?

Answer using your own words and observations from the programs. You do not need to use Big O notation.

7. Final task and submission

Complete all four tasks using either C++ or Python. Create one folder named with your Student ID and Name. Include:

- SequentialSearch.cpp or SequentialSearch.py
- BinarySearch.cpp or BinarySearch.py
- One PDF lab report with program-output screenshots, the manual tracing table, results from Task 4, and answers to all eight questions.

Submit the completed folder to your class monitor before the end of this class.

All code must be typed manually. Copying and pasting code is not allowed. Students should understand their programs and be ready to explain how each search works.